

KNN para regressão

Olá! Nesta última videoaula sobre KNN, iremos falar um pouco sobre o KNN para regressão. O KNN para regressão é bem parecido com o KNN para classificação. Poucas mudanças serão aplicadas.

Primeiro, a intuição do KNN para regressão é um pouco diferente, visto que não iremos mais escolher os vizinhos mais próximos e pegar a classe da maioria dos vizinhos mais próximos. No caso da regressão, iremos efetuar uma média do rótulo dos vizinhos mais próximos. Ou seja, o processo de cálculo da distância é o mesmo, porém, na hora de aplicar o rótulo do novo elemento, iremos utilizar apenas a média dos rótulos dos vizinhos mais próximos. Então, a única diferença seria essa entre os dois modelos.

Na implementação, também temos uma pequena diferença que seria no modelo classificado. Iremos, primeiramente, importar o modelo do KNN para regressão. Neste modelo, estamos utilizando o KNN para regressão. A linha ficou: "from sklearn.neighbors import KNeighborsRegressor..."

Executando esta linha, teremos importado o KNN para regressão.

E para utilizá-lo, iremos instanciá-lo na variável KNN, da mesma maneira que fizemos anteriormente: "knn = KNeighborsRegressor". O "KNeighborsRegressor" segue a mesma estratégia de escrita do "KNeighborsClassifier", ou seja, as letras K, N e R são maiúsculas.

Continuando: "...KNeighborsRegressor (n_neighbors=3)".

Para dizer que o rótulo do novo elemento será igual a média dos três vizinhos mais próximos. Executando essa linha, teremos o KNN implementado. Para executar alguma predição diferente neste KNN, iremos utilizar um "dataset", um conjunto de dados diferente do que vínhamos utilizando até então, pois o conjunto de dados que estávamos utilizando era usado para classificação.

Agora, tentaremos utilizar outro conjunto de dados para regressão. E esse conjunto de dados é referente ao preço de casas de diferentes regiões de Boston. Então, iremos importar esse conjunto de dados direto do próprio sklearn.

O sklearn tem alguns conjuntos de dados disponíveis em seu pacote. E esse conjunto de dados de preços de casas em Boston é um deles. Então, iremos utilizar:

```
from sklearn.datasets import load_boston.
```

Ao executar esta linha, teremos importado o nosso "dataset" de Boston. E para colocá-lo em uma variável, iremos utilizar o seguinte comando: `data= load_boston()`.

Dessa maneira, teremos todo o conjunto de dados armazenado na variável "data".

E essa variável "data" é um dicionário de Python que possui várias informações armazenadas. Para facilitar, iremos importar esse conjunto de dados de uma maneira diferente que irá separar os dados em atributos e rótulos. Então, iremos utilizar: `X, y = load_boston(return_X_y=True)`. Dessa maneira, essa função "load_boston" irá retornar tanto os atributos quanto os rótulos já separados.

Se imprimirmos o "X", teremos os atributos que são 13 colunas, utilizando "X.shape". Teremos 506 linhas e 13 colunas. E "y.shape", que são 506 rótulos que seriam os preços das casas de cada região a média do preço das casas de cada região.

Então, se você quer entender melhor como funciona essa dataset, você pode utilizar o seguinte comando: `print(load_boston()[0]'DESCR'])`. Esse "DESCR" seria description ou descrição do conjunto de dados.

Ao executar esta linha, temos informações impressas na tela sobre esse dataset como o número de instâncias, que são 506 elementos o número de atributos – 13, e o valor médio do preço da casa, que é o 14º atributo, que seria o rótulo e a descrição de cada um desses atributos. Por exemplo, um dos atributos é a criminalidade per capita da região proporção de zonas que possuem lotes acima de 25.000m² à proporção de indústrias, vários outros atributos deste conjunto de dados. Então, nós já temos o conjunto de dados separado em X, que são os atributos e y, os rótulos, e iremos efetuar a separação em conjunto de treino e teste. Logo, iremos importar a separação em treino e teste: `from sklearn.model_selection import train_test_split`. Ao acionar Shift + Enter, nós executamos essa célula e já criamos uma próxima.

Temos a função de divisão em treino e teste. Iremos executar essa função na próxima linha que é descrita da seguinte maneira: `X_train, X_test, y_train, y_test =`

`train_test_split(X,y)`: da maneira como efetuamos a leitura dos arquivos de Boston na função `"load_boston"`. Rodando esta função, ocorre que a divisão em treino e teste é efetuada.

A partir desse momento, a execução do KNN para regressão é igual a execução do KNN para classificação. Onde teremos: `knn.fit(X_train, y_train)`.

Ao executar, temos que o KNN para regressão armazenou os valores de train. E com: `knn.score (X_test, y_test)`, temos o resultado da regressão em cima desse conjunto de dados que deu igual a 0,57. Então, até como um exercício, eu gostaria que vocês tentassem implementar também neste conjunto de teste a utilização do Min e Max Scaler para mudar a escala dos atributos e verificar se melhora ou não o resultado. Escalando os atributos, podemos atingir um resultado melhor do KNN para regressão da mesma maneira que fizemos também para o KNN para classificação. E dessa maneira, nós encerramos a aula sobre KNN utilizando o mínimo de pré-processamento para melhorar o resultado escolhendo um número de vizinhos e executando o teste. Então, peço que vocês possam testar outras quantidades de vizinhos e efetuar o pré-processamento das features para estarem sempre em uma escala menor. Utilizando, por exemplo, o Min e Max Scaler.

É isso! Até mais!